

REC

Department of Computer Science and Business Systems (CSBS)

A Comparative Study of Low-Code Development versus Traditional Application Development Approaches

Subject: Low-Code Application Development using Zoho Creator

Name	SRI JANANI B
Roll No.	241401108
Department	CSBS (Computer Science and Business Systems)
College	REC
Subject	Low-Code Application Development using Zoho Creator

1. Introduction

Building software applications is one of the most important tasks in any organisation today. Over the years, two main approaches have become popular for developing applications: the traditional (custom-code) approach and the modern low-code approach. Both methods aim to solve the same problem - creating working software - but they differ greatly in the way they do it, the skills they need, the cost involved and the time they take.

Traditional software development means writing every part of the application manually using programming languages such as Java, Python or C++. It gives full control and flexibility but requires trained developers, more time and a bigger budget.

Low-code development, on the other hand, uses visual, drag-and-drop tools to build applications with very little manual coding. Platforms such as Zoho Creator, Mendix and Appian allow even non-technical users to design, build and deploy working applications quickly. This study compares both approaches in detail and explains where each one fits best, with special focus on Zoho Creator as a leading low-code platform.

1.1 Objective of the Study

- To understand the meaning and working of traditional and low-code software development.
- To compare both approaches on the basis of cost, time, skill requirement, flexibility and maintenance.
- To study Zoho Creator as an example of a modern low-code platform.
- To identify the situations where each approach is the better choice for a business.

1.2 What is Traditional Software Development?

Traditional software development is the classic way of building applications. Developers write source code line by line using programming languages and frameworks. This method uses many different tools for design, coding, testing and deployment. Because everything is built from scratch, developers need strong technical knowledge. This approach gives complete control over how the application looks and works, but it also takes more time, more money and a skilled team to finish the project.

1.3 What is Low-Code Software Development?

Low-code software development is a modern approach where applications are built mostly through visual interfaces instead of hand-written code. It provides ready-made building blocks, templates and drag-and-drop components so that users can assemble an application quickly. This approach reduces the need for deep programming knowledge and allows both developers and business users to create functional applications in a much shorter time. Zoho Creator is one of the most popular low-code platforms used for this purpose.

2. Background of the Study

Software development has evolved a great deal over the last few decades. In the early days, every application was built completely from scratch using low-level programming languages, which demanded a large team of specialists and a long development cycle. As businesses grew and the demand for digital tools increased, the need for faster and cheaper ways to build software became very important.

This need gave rise to the low-code movement. Gartner and other research firms have reported that a large and growing share of new business applications are now being built using low-code or no-code platforms, because organisations want to reduce their dependence on scarce, expensive developer resources while still keeping up with fast-changing business needs.

At the same time, traditional software development has not disappeared. It remains the backbone of large-scale systems such as banking platforms, operating systems, telecom infrastructure and other software where performance, security and full customisation are critical. This study looks at both approaches side by side so that the differences, and the situations where each works best, become clear.

2.1 Scope of the Study

This report focuses on comparing the two development approaches from the point of view of a student and a small-to-medium business. It covers the working principle, cost, time, required skill, maintenance and typical use cases of both approaches, and uses Zoho Creator as a real example of a low-code platform in action.

2.2 Methodology

The comparison in this report is based on a study of the working principles of both approaches, supported by common industry observations and publicly available material on low-code and traditional development, including a review of the practical features offered by the Zoho Creator platform.

3. Detailed Comparison

The table below compares traditional software development and low-code software development on several important factors such as skill requirement, cost, bugs, maintenance, tools used, platform dependency, and suitable use cases.

S.No.	Traditional Software Development	Low-Code Software Development
1	Uses many tools and functions. Needs strong technical and coding skills to build and run applications.	Uses a small set of simple tools. Even non-technical staff can build applications without deep coding skills.

S.No.	Traditional Software Development	Low-Code Software Development
2	Because the process is complex, it often produces more bugs, making the app harder to use smoothly.	Produces fewer bugs during app building, so the apps run smoothly with less hassle.
3	Quite expensive to build. It needs a lot of time, manpower and resources, which raises the overall budget.	Much cheaper than traditional development. The company pays only for the service it uses, not for building everything from scratch.
4	Best suited for large companies that need highly customised software which cannot be built with low-code tools.	Best suited for small and medium businesses that need attractive apps built quickly with less coding.
5	Has a complicated setup and configuration, so it is less flexible. Learning the code properly takes a long time.	Easy to use because of drag-and-drop features. There is little or no need to write code, so app screens can simply be designed visually.
6	Maintenance is difficult. Any update or change needs an in-house or third-party team to plan, code, test and train users.	Maintenance is easier because the platform provider (APAAS - Application Platform as a Service) handles most updates and enhancements.
7	Developers use programming languages and web frameworks to build fully coded web applications.	Tools such as Zoho Creator, Mendix and Appian are used to quickly build low-code web applications.
8	No platform limits - the application can be deployed on any platform the developer chooses.	Deployment is fast but limited only to the platforms supported by the specific low-code tool being used.
9	Highly adaptable, very fast performance, extremely scalable, but debugging and testing take more time.	Slightly less flexible, good speed and efficiency, live debugging keeps apps mostly error-free, but scalability is limited.
10	Development cost is high because of the time, tools and skilled manpower needed.	Development cost is low compared to the traditional approach.
11	Preferred for open-ended, unrestricted solutions and apps that need a high level of special interactivity.	Preferred for fast enterprise transformation, adaptable solutions, and lets business users build apps quickly.

Table 1: Traditional Development vs Low-Code Development

4. Factor-wise Summary

Apart from the point-by-point comparison above, it helps to look at a short summary of the two approaches across the factors that matter most to a business: speed, cost, skill needed, control, and long-term maintenance.

4.1 Speed of Development

Low-code platforms are much faster because screens, forms and workflows are assembled visually instead of being coded line by line. A working prototype that may take weeks in traditional development can often be built in a few days using a low-code tool.

4.2 Cost of Development

Traditional development needs a bigger team of skilled programmers working over a longer period, which increases the overall cost. Low-code development reduces cost because fewer developers are needed and the platform itself takes care of many technical tasks such as hosting, security patches and basic testing.

4.3 Skill Requirement

Traditional development demands strong knowledge of programming languages, data structures, and software design. Low-code development lowers this barrier, allowing business analysts and citizen developers, not just professional programmers, to build usable applications.

4.4 Control and Customisation

Traditional coding gives complete freedom to design every feature exactly as required, which is important for large, unique, or highly specialised systems. Low-code platforms provide strong customisation for common business needs, but very unusual or highly complex features may be difficult or impossible to build without extra scripting.

4.5 Maintenance and Upgrades

In traditional development, the organisation's own team is fully responsible for fixing bugs, applying security updates and testing every change. In low-code development, the platform vendor manages most of the underlying maintenance, so the business only needs to update the app logic when its own requirements change.

5. Advantages and Disadvantages

The points below summarise the main strengths and weaknesses of each approach in a simple, easy-to-read format.

5.1 Traditional Software Development

Advantages	Disadvantages
Complete control over design, structure and functionality.	Takes a long time to build and launch the application.
Can be built using any programming language or framework.	Requires highly skilled and often expensive developers.

Advantages	Disadvantages
Highly scalable for large and complex systems.	Maintenance and updates are time-consuming and costly.
No dependency on a third-party platform vendor.	Higher chance of human error and bugs during manual coding.
Best performance since every line of code can be optimised.	Not ideal for small projects with tight budgets or deadlines.

Table 2: Advantages and Disadvantages of Traditional Development

5.2 Low-Code Software Development

Advantages	Disadvantages
Much faster development using drag-and-drop tools.	Limited customisation for very unique requirements.
Lower cost since fewer developers and less time are needed.	Scalability can be restricted for very large systems.
Easy for non-technical business users to learn and use.	Dependent on the platform vendor for updates and support.
Maintenance is simplified as the vendor manages the platform.	Deployment is limited to platforms supported by the tool.
Built-in security, templates and reusable components.	May involve subscription or licensing costs over time.

Table 3: Advantages and Disadvantages of Low-Code Development

6. Low-Code Development using Zoho Creator

Zoho Creator is a widely used low-code application development platform that allows users to build custom business applications using a simple drag-and-drop builder. It provides pre-built templates, form builders, workflow automation and reporting tools, so that even users with little programming background can create working applications for tasks such as inventory management, CRM, HR management and approval workflows.

With Zoho Creator, developers can still write custom scripts (using Deluge, its built-in scripting language) when they need extra control, which makes it a good middle ground between pure low-code simplicity and traditional coding flexibility. This makes Zoho Creator a good example of how low-code platforms are closing the gap with traditional development while still keeping things simple and fast for everyday business needs.

6.1 Key Advantages and Limitations

Low-code platforms like Zoho Creator offer speed, ease of use, lower cost, simplified maintenance and quick automation of repetitive tasks. However, they can have limited customisation, restricted scalability

for very large systems, fewer integration options in some cases, and a dependency on the platform vendor for updates and long-term support.

Traditional development continues to offer complete control, unlimited customisation, strong scalability and the freedom to use any programming language or framework. Its main drawbacks are the higher cost, longer development time, the need for skilled developers and a more complex maintenance process.

6.2 Core Features of Zoho Creator

- Drag-and-drop form builder to design data entry screens without coding.
- Deluge scripting language for adding custom logic and advanced automation when needed.
- Built-in workflow automation to handle approvals, notifications and reminders.
- Ready-made templates for common business needs such as inventory, CRM and HR.
- Multi-platform access, allowing apps to run on web browsers and mobile devices.
- Built-in reports and dashboards to visualise business data instantly.

7. Case Study: Building a Leave Management App in Zoho Creator

To understand low-code development in practice, this section walks through how a simple Employee Leave Management application can be built using Zoho Creator, and compares the effort with what the same application would need in traditional development.

7.1 Steps Followed in Zoho Creator

- Step 1: Create a new application and add a form to collect employee details such as name, department and employee ID.
- Step 2: Add a leave request form with fields for leave type, start date, end date and reason, using drag-and-drop form elements.
- Step 3: Set up a workflow rule so that a leave request is automatically sent to the reporting manager for approval.
- Step 4: Use Deluge scripts to calculate the number of leave days automatically and update the employee's leave balance.
- Step 5: Build a simple report and dashboard to show pending, approved and rejected leave requests.
- Step 6: Publish the application and share access with employees and managers on web and mobile.

7.2 Comparison with Traditional Development

Building the same leave management application using traditional development would require a front-end interface built with a framework, a back-end server to handle business logic, a database to store

employee and leave data, and a hosting setup to make it accessible. This would typically take a team of developers several weeks, compared to the few days it takes to build the same working application in Zoho Creator using its visual builder and built-in workflow engine.

This case study shows clearly why low-code platforms are attractive for common, repeatable business applications, while also showing that traditional development would still be preferred if the organisation needed very specific features, such as a highly customised leave policy engine integrated deeply with other in-house systems.

8. Future Trends in Software Development

The gap between low-code and traditional development is expected to keep narrowing in the coming years. Several trends are shaping this change.

- **AI-assisted development:** Both low-code platforms and traditional coding tools are adding AI features that generate code, forms and workflows automatically from simple text descriptions.
- **Hybrid platforms:** Low-code tools are increasingly allowing developers to insert custom code or extend the platform with APIs, blurring the line between low-code and traditional coding.
- **Citizen development:** More non-technical employees are being trained to build their own small applications using low-code tools, reducing the workload on central IT teams.
- **Enterprise-grade low-code:** Vendors are improving scalability, security and integration options in low-code platforms, making them suitable for larger and more critical applications than before.

These trends suggest that, rather than one approach replacing the other, most organisations will continue to use a thoughtful mix of low-code and traditional development, matched to the needs of each project.

9. When to Choose Which Approach

Choosing between low-code and traditional development should depend on the nature of the project rather than personal preference alone. The points below can act as a simple decision guide for students and businesses.

9.1 Choose Traditional Development When

- The application needs a highly unique feature that no low-code platform currently supports.
- The project is very large in scale and expects millions of users with heavy performance needs.
- The organisation already has an experienced, in-house development team.
- Full ownership of the source code and complete independence from any vendor is required.

9.2 Choose Low-Code Development When

- The application must be delivered quickly, within days or a few weeks.
- The budget for the project is limited.
- Business users, not just professional developers, need to build or modify the app.
- The requirement matches common business needs such as forms, approvals, dashboards and inventory tracking.

9.3 A Hybrid Approach

In practice, many modern organisations combine both approaches. They use low-code platforms such as Zoho Creator for internal tools, quick prototypes and department-level applications, while relying on traditional development for their core, mission-critical products. This hybrid strategy allows a business to move fast without giving up control where it truly matters.

10. Conclusion

Both traditional and low-code software development have their own strengths. Traditional development is the right choice when an organisation needs a highly customised, large-scale and unique application, and has the budget and skilled team to support it. Low-code development, especially through platforms like Zoho Creator, is the right choice when speed, cost saving and ease of use are more important, such as for small businesses, internal tools and quick enterprise applications.

This study has shown that low-code development, using tools like Zoho Creator, offers real benefits in speed, cost and ease of use, especially for common business applications. At the same time, traditional development remains essential for large-scale, highly specialised and performance-critical systems. In short, there is no single 'best' approach - the right choice depends on the size of the project, the budget available, the time available and the technical skill of the team. Many organisations today use a mix of both approaches, using low-code platforms for fast, simple applications and traditional development for complex, mission-critical systems.